

XIMOD Architect

Benutzerhandbuch

Ein Tool zur Erstellung von FOMOD-Installationsprogrammen für Bethesda-Spiel-Mods

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 1.0.0

Plattformübergreifend: Windows · Linux · macOS



Inhalt

1. Überblick3

- 1.1 Was ist XIMOD Architect?3
- 1.2 Das FOMOD-Format3
- 1.3 Installation und Anforderungen4
- 1.4 Dateistruktur4

2. Erste Schritte6

- 2.1 Starten der Anwendung6
- 2.2 Übersicht über die Benutzeroberfläche6
- 2.3 Ein neues Projekt erstellen6
- 2.4 Ein Projekt öffnen und speichern6
- 2.5 Erster Start7
- 2.6 Freie Fenster und Tastaturnavigation7

3. Die Menüs8

- 3.1 Menü „Datei“8
- 3.2 Menü „Optionen“8
- 3.3 Menü „Hilfe“9

4. Funktionen im Detail10

- 4.1 Die Registerkarte „Mod-Info“10
- 4.2 Die Registerkarte „Installationsschritte“10
- 4.3 Die Registerkarte „Erforderliche Installationen“12
- 4.4 Die Registerkarte „Bedingte Installationen“12
- 4.5 Das Fenster „Einstellungen“12
- 4.6 Skripte vor und nach dem Speichern13
- 4.7 Das Fenster „Info“14
- 4.8 Erstellen der FOMOD-Dateien14
- 4.9 Übersetzungseditor15
- 4.10 XML-Editor16
- 4.11 Schriftartenverwaltung17
- 4.12 Zwei FOMOD-Dateien zusammenführen17
- 4.13 Exportieren des Distributionsarchivs18
- 4.14 Das Fenster „Eigenschaften“18
- 4.15 Befehlszeilenmodus (CLI)19

5. FOMOD-Tag-Referenz20

- 5.1 Die Datei „info.xml“20
- 5.2 Die Datei „ModuleConfig.xml“20

6. Die Datei „ModuleConfig.xml“ im Detail23

- 6.1 Wie das Installationsprogramm die Datei ausliest23
- 6.2 Der Stamm und der Header23
- 6.3 Dateien installieren23
- 6.4 Schritte, Gruppen und Optionen24
- 6.5 Der Typ einer Option und ihr Verhalten25
- 6.6 Flags, Abhängigkeiten und bedingte Installationen25

7. Beispiele für „ModuleConfig.xml“27

- 7.1 Minimalbeispiel – erforderliche Dateien27
- 7.2 Ausschließliche Auswahl – SelectExactlyOne27
- 7.3 Bedingte Installation – Flags28

Anhänge29

- Anhang A – Die Datei „Config.ini“29

Anhang B – Tastaturkürzel	29
Anhang C – Glossar	30
Anhang D – Vollständiges FOMOD-Beispiel	32
D.1 „info.xml“	32
D.2 „ModuleConfig.xml“	32
Anhang E – Lizenz und Danksagungen	36

1. Überblick

1.1 Was ist XIMOD Architect?

XIMOD Architect ist ein plattformübergreifendes Tool zur Erstellung von FOMOD-Installationsprogrammen für Bethesda-Spiel-Mods (Skyrim, Fallout, Starfield, Oblivion, Morrowind usw.). Damit können Sie visuell einen vollständigen Installationsassistenten erstellen – Schritte, Optionsgruppen, Plugins, bedingte Dateien – und anschließend automatisch die standardisierten XML-Dateien generieren, die von Mod-Managern interpretiert werden können.

Die Anwendung unterstützt elf Spiele, die in der Datei „Categories.json“ definiert sind: die drei Editionen von Skyrim (Legendary, Special Edition und VR), Fallout 3, Fallout: New Vegas, Fallout 4 und Fallout 4 VR, Oblivion und Oblivion Remastered, Morrowind sowie Starfield. Da es sich um eine externe und bearbeitbare Datei handelt, lässt sich die Liste erweitern, ohne dass die Anwendung neu kompiliert werden muss.

Die Software ist eine Neuprogrammierung des ursprünglichen FOMODCreationTool in der Programmiersprache Rust. Diese Überarbeitung sorgt für eine bessere Leistung, ein identisches Verhalten unter Windows, Linux und macOS sowie eine moderne Benutzeroberfläche. XIMOD Architect zeichnet sich zudem durch sein sehr fortschrittliches Internationalisierungssystem aus: Die Benutzeroberfläche kann in eine Vielzahl von Sprachen übersetzt werden, wobei die für das jeweilige Schriftsystem geeigneten Schriftarten dynamisch geladen werden.

Die wichtigsten Funktionen sind wie folgt:

- Erstellung vollständiger FOMOD-Installationspakete;
- mehrstufige Installationsassistenten;
- Plugin-Gruppen mit verschiedenen Auswahltypen (genau eines, mindestens eines, beliebig viele usw.);
- bedingte Dateiinstallation basierend auf den Auswahlentscheidungen des Benutzers;
- Abhängigkeitsmuster, die den Typ eines Plugins bestimmen;
- ein System von „Flags“ (Bedingungsflags), die den Ablauf der Installation steuern;
- benutzerdefinierte Skripte, die vor oder nach dem Speichern ausgeführt werden;
- eine mehrsprachige Benutzeroberfläche und dynamische Schriftarten;
- ein nativer, transparenter Startbildschirm.

1.2 Das FOMOD-Format

FOMOD („Fallout Mod“) ist das Standardformat für Mod-Installationsprogramme für Bethesda-Spiele. Ein FOMOD-Installationsprogramm basiert auf zwei XML-Dateien, die in einem Unterordner namens „fomod“ gespeichert sind:

- `info.xml` – die Mod-Metadaten: Name, Autor, Version, Website, Beschreibung, Kategorie.
- `ModuleConfig.xml` – die Installationskonfiguration: Schritte, Gruppen, Plugins, Dateien, Bedingungen.

Die von XIMOD Architect erstellten Dateien sind mit den wichtigsten Mod-Managern kompatibel:

- Mod Organizer 2 (MO2);
- Vortex;

- Nexus Mod Manager (NMM).

Hinweis – Der Name des „fomod“-Ordners sowie die Namen „info.xml“ und „ModuleConfig.xml“ sind durch die Spezifikation vorgegeben und werden von den Mod-Managern unverändert erkannt. Sie dürfen niemals umbenannt werden.

1.3 Installation und Anforderungen

Systemanforderungen (Richtwerte):

Punkt	Empfehlung
System	Windows 10 oder höher, Linux (aktuelle glibc) oder macOS 11 oder höher.
Bildschirm	Grafikkarte, die mit OpenGL 3.3 kompatibel ist (eguis „glow“-Renderer).
Arbeitsspeicher	Einige hundert MB – die Anwendung ist ressourcenschonend.
Speicherplatz	Einige Dutzend MB, abhängig von den Noto-Schriftarten und den in assets/ bereitgestellten Flags.

Windows

Legen Sie die ausführbare Datei „ximod-architect.exe“, den Ordner „assets“ (Daten, Sprachpakete, Schriftarten, Bilder, Symbole) und optional die Datei „assets/images/splash.png“ für den Startbildschirm im selben Ordner ab. Das Anwendungssymbol wird beim Kompilieren in die ausführbare Datei eingebettet. Führen Sie anschließend „ximod-architect.exe“ aus.

Linux

Ein Installationsskript ist im Verzeichnis „packaging/linux“ enthalten:

```
# Systemweite Installation (erfordert sudo)
sudo ./packaging/linux/install.sh

# Installation für den aktuellen Benutzer
./packaging/linux/install.sh --user
```

Die Kompilierung aus dem Quellcode erfordert Rust 1.70 oder höher sowie unter Linux die folgenden Entwicklungsbibliotheken: libgtk-3-dev, libxcb-render0-dev, libxcb-shape0-dev, libxcb-xf86dev, libspeechd-dev, libxcbcommon-dev, libssl-dev und libx11-dev.

macOS

Ein Skript erstellt ein verteilungsfertiges Anwendungs-Bundle (.app):

```
./packaging/macos/create-bundle.sh
```

Das Bundle „XIMOD Architect.app“ wird im Ordner „dist“ erstellt. Anschließend kann mit dem Dienstprogramm „hdiutil“ ein Disk-Image (.dmg) erstellt werden.

1.4 Dateistruktur

Die zur ausführbaren Datei gehörende Dateistruktur sieht wie folgt aus:

ximod-architect(.exe)	→ ausführbare Datei
Config.ini	→ Konfiguration (wird beim ersten Start erstellt)
assets/	
data/	
Categories.json	→ Spiele und Nexus-Kategorien
Languages.json	→ 6.777 Sprachen (Endonym, ISO-Codes, Schriftart)
Countries.json	→ 244 Länder (Namen, Flagge, Amtssprachen)
CountryLanguages.json	→ Sprachen, die in den einzelnen Ländern gesprochen werden
locales/	→ Übersetzungen (ein Ordner pro Sprache)
eng/main.ftl	
fra/main.ftl	
...	→ 32 Sprachen verfügbar
fonts/	→ Noto-Schriftarten (eine pro Schriftsystem)
icons/	→ Anwendungssymbole
images/	
splash.png	→ Begrüßungsbildschirm
svg/	→ Landesflaggen (244 SVG-Dateien)

2. Erste Schritte

2.1 Starten der Anwendung

Beim Start zeigt XIMOD Architect zunächst einen nativen, transparenten Startbildschirm mit einem Einblendeeffekt an, dessen Dauer einstellbar ist (standardmäßig zwei Sekunden; der Bildschirm kann deaktiviert werden). Anschließend öffnet sich das Hauptfenster, zentriert auf dem primären Bildschirm. Seine Anfangsgröße wird aus der Konfigurationsdatei übernommen (standardmäßig 1280 × 800) und darf nicht kleiner als 800 × 600 sein. In der Titelleiste wird der Name der Anwendung gefolgt von der Versionsnummer angezeigt.

2.2 Übersicht über die Benutzeroberfläche

Das Hauptfenster besteht aus drei Bereichen:

- **die Menüleiste** oben: „Datei“, „Optionen“ und „Hilfe“ (siehe Kapitel 3);
- **dem Registerbereich** in der Mitte, der die vier Arbeitsbereiche vereint: Mod-Info, Installationsschritte, Erforderliche Installationen und Bedingte Installationen (siehe Kapitel 4);
- **Die Statusleiste** am unteren Rand: links die aktuelle Meldung (beim Start: „Bereit“); rechts, wenn ein Projekt ungespeicherte Änderungen enthält, ein gelber Punkt, gefolgt vom Text „Geändert“.

Das Erscheinungsbild der Benutzeroberfläche (dunkles, helles oder System-Design, Schriftgröße, Sprache) wird im Fenster „Einstellungen“ festgelegt, das in § 4.5 beschrieben wird.

2.3 Ein neues Projekt erstellen

Das Erstellen eines Installationsprogramms erfolgt im Allgemeinen in den folgenden Schritten:

- **Neues Projekt** – Menü „Datei“ → „Neu“, dann das Stammverzeichnis auswählen, das die Dateien Ihres Mods enthält (Schaltfläche „Durchsuchen...“ auf der Registerkarte „Mod-Info“).
- **Informationen eingeben** – Geben Sie auf der Registerkarte „Mod-Info“ den Mod-Namen, den Autor, die Version, das Spiel, die Kategorie, die Website, das Titelbild und die Beschreibung ein.
- **Assistenten erstellen** – Fügen Sie auf der Registerkarte „Installationsschritte“ zunächst Schritte, dann Optionsgruppen und anschließend Plugins hinzu.
- **Dateien zuordnen** – Fügen Sie für jedes Plugin die zu installierenden Dateien oder Ordner hinzu; das Zielverzeichnis wird automatisch berechnet.
- **Bedingungen festlegen** – Verwenden Sie Flags und bedingte Installationen, um die Installation an die Auswahl des Benutzers anzupassen.
- **Speichern** – Menü „Datei“ → „Speichern“ (Strg+S): XIMOD Architect erstellt den Ordner „fomod“ und schreibt die Dateien „info.xml“ und „ModuleConfig.xml“ darin ab.

2.4 Projekt öffnen und speichern

Beim Speichern wird immer der Unterordner „fomod“ im Stammverzeichnis erstellt und die beiden XML-Dateien werden darin im UTF-8-Format abgelegt. Der Befehl „Speichern“ bleibt deaktiviert, solange kein Stammverzeichnis definiert wurde.

Es stehen zwei Öffnungsmodi zur Verfügung. „Ordner öffnen...“ fragt nach dem Stammverzeichnis und lädt das Projekt aus dessen „fomod“-Unterordner. „Datei öffnen...“ fragt direkt nach einer XML-Datei: Das Stammverzeichnis wird dann abgeleitet (es wird davon ausgegangen, dass sich die Datei in einem

„fomod“-Unterordner befindet). Beim Einlesen erkennt XIMOD Architect automatisch die Dateikodierung (UTF-8, mit oder ohne Byte-Order-Mark, sowie UTF-16), wodurch es möglich ist, von anderen Tools erstellte Installationsprogramme erneut zu öffnen.

Jedes erfolgreich geöffnete Projekt wird an den Anfang der Liste der zuletzt verwendeten Dateien gesetzt, die über „Datei → Zuletzt verwendet“ aufgerufen und auf der Registerkarte „Zuletzt verwendete Dateien“ in den Einstellungen verwaltet werden kann.

2.5 Erster Start

Beim allerersten Start startet XIMOD Architect auf Englisch und öffnet automatisch das Fenster „Einstellungen“, damit der Benutzer sein Land und anschließend seine Sprache auswählen kann.

Dieses Verhalten wird durch den Schlüssel „FirstStart“ in der Datei „Config.ini“ gesteuert: Er ist auf 0 gesetzt, solange die Erstkonfiguration noch nicht gespeichert wurde, und wechselt auf 1, sobald zum ersten Mal auf „Speichern“ geklickt wird. Bei nachfolgenden Starts wird das Programm dann direkt in der gewählten Sprache gestartet.

Wenn der Benutzer das Fenster mit „Abbrechen“ schließt, bleibt der Schlüssel auf 0 gesetzt und das Fenster wird beim nächsten Start erneut geöffnet.

2.6 Freie Fenster und Tastaturnavigation

Mehrere Werkzeuge öffnen sich in **freien (unabhängigen) Fenstern**: die „Länderauswahl“, das Fenster „Eigenschaften“, der Übersetzungseditor und der XML-Editor. Jedes lässt sich frei verschieben und in der Größe ändern, auch auf einem zweiten Bildschirm, und arbeitet parallel zum Hauptfenster.

Zentrierung und Speicherung — beim ersten Öffnen wird ein freies Fenster auf dem XIMOD-Hauptfenster zentriert. Wird es verschoben oder in der Größe geändert, werden Position und Größe beim Schließen (und beim Beenden der Anwendung) gespeichert und beim nächsten Öffnen wiederhergestellt. Diese Werte werden in Config.ini im Abschnitt [WindowPositions] abgelegt (siehe Anhang A). Das Hauptfenster merkt sich seine Größe, aber nicht seine Position: Es bleibt bei jedem Start zentriert.

Schließen mit Esc — die Esc-Taste schließt das aktive freie Fenster sowie die modalen Dialoge des Hauptfensters (Einstellungen, Über, Bestätigung, Prüfbericht, Skripte); bei einer Bestätigung wirkt Esc wie „Abbrechen“ und führt die Aktion nie aus. Zwei Ausnahmen: der Übersetzungseditor (um ein versehentliches Schließen während der Eingabe zu vermeiden) und das Hauptfenster schließen sich nicht mit Esc.

Tastaturnavigation — die Listen im Fenster „Eigenschaften“, die Tabelle des Übersetzungseditors und das Flaggenraster der Länderauswahl lassen sich vollständig über die Tastatur bedienen: Pfeiltasten (und der Ziffernblock), Bild auf/Bild ab, Pos1/Ende, Eingabe sowie Tab/Umschalt+Tab. Die Einzelheiten stehen in den jeweiligen Abschnitten und sind in Anhang B zusammengefasst.

Filter löschen — jedes Feld „Filter:“ besitzt rechts ein Löschesymbol (rotes Kreuz), das den Filter mit einem Klick leert.

3. Die Menüs

Die Menüleiste enthält drei Menüs: „Datei“, „Optionen“ und „Hilfe“. Die angezeigten Tastenkombinationen sind auch systemweit aktiv (die Strg-Taste entspricht der Cmd-Taste unter macOS). Menüs und Tastenkombinationen werden ignoriert, solange ein modales Fenster geöffnet ist.

3.1 Menü „Datei“

Befehl	Tastenkombination	Beschreibung
Neu	Strg+N	Setzt das Projekt zurück. Wenn ungespeicherte Änderungen vorliegen, wird zunächst eine Bestätigung angefordert.
Ordner öffnen...	Strg+O	Wählt ein Stammverzeichnis aus und lädt das Projekt aus dessen „fomod“-Unterordner.
Datei öffnen...	Strg+Umschalt+O	Wählt direkt eine XML-Datei aus; das Stammverzeichnis wird automatisch ermittelt.
Speichern	Strg+S	Erzeugt die Dateien „fomod/info.xml“ und „fomod/ModuleConfig.xml“. Diese Funktion ist deaktiviert, solange kein Stammverzeichnis definiert ist.
FOMOD zusammenführen...	—	Fügt die Schritte, erforderlichen Dateien und bedingten Installationen eines anderen FOMOD zum aktuellen Projekt hinzu. Deaktiviert, solange kein Projekt geöffnet ist. Siehe § 4.12.
Distributionsarchiv exportieren...	—	Schreibt das FOMOD und komprimiert anschließend das gesamte Stammverzeichnis in ein .zip-Archiv, das zur Veröffentlichung bereit ist. Deaktiviert, solange kein Stammverzeichnis definiert ist. Siehe § 4.13.
Zuletzt verwendet	—	Untermenü mit einer Liste der zuletzt geöffneten Projekte. Zeigt „(leer)“ an, wenn die Liste leer ist.
Beenden	Strg+Q	Schließt die Anwendung. Die Konfiguration wird beim Beenden gespeichert.

3.2 Menü „Optionen“

Befehl	Tastenkombination	Beschreibung
Einstellungen	Strg+,	Öffnet das Fenster „Einstellungen“ (Land, Sprache, Design, Schriftart usw.). Siehe Abschnitt 4.5.
Skript vor dem Speichern...	—	Bearbeitet das Skript, das vor dem Speichern ausgeführt wird. Siehe § 4.6.
Skript nach dem Speichern...	—	Bearbeitet das Skript, das nach dem Speichern ausgeführt wird. Siehe § 4.6.

Befehl	Tastenkombination	Beschreibung
Übersetzung...	—	Öffnet den Editor für die Übersetzung der Benutzeroberfläche. Siehe § 4.9.
XML-Editor	—	Untermenü, das Zugriff auf „info.xml“ und „ModuleConfig.xml“ gewährt. Siehe § 4.10.
FOMOD validieren	—	Überprüft das Projekt und die Konformität von „ModuleConfig.xml“ mit dem ModConfig 5.0-Schema und zeigt anschließend einen detaillierten Bericht an. Siehe § 4.8.
Eigenschaften...	—	Öffnet den schreibgeschützten Explorer der Länder-/Sprachdatenbank. Siehe § 4.14.

3.3 Menü „Hilfe“

Befehl	Tastenkombination	Beschreibung
Über	F1	Zeigt das Fenster „Über“ an (Name, Version, Lizenz). Siehe § 4.7.

4. Funktionen im Detail

4.1 Die Registerkarte „Mod-Info“

Auf dieser Registerkarte werden die Mod-Metadaten und die Auswahl des Stammverzeichnisses zusammengefasst.

Arbeitsbereich – In der Zeile „Stammverzeichnis:“ wird der Arbeitsordner angezeigt (oder „(Verzeichnis auswählen)“, falls noch kein Verzeichnis festgelegt wurde). Über die Schaltfläche „Durchsuchen...“ können Sie das Verzeichnis auswählen.

Die Metadatenfelder lauten dann wie folgt:

Feld	Rolle
Mod-Name	Angezeigter Name des Installationsprogramms (erforderlich).
Autor	Name des Mod-Autors.
Version	Versionsnummer (standardmäßig 1.0.0 für ein neues Projekt).
Spielname	Zielspiel, ausgewählt aus einer Liste, die aus der Datei „Categories.json“ stammt.
Kategorie	Kategorie des Mods. Hängt vom ausgewählten Spiel ab; andernfalls wird eine Standardliste angeboten (Animation, Rüstung, Audio, Texturen, Waffen usw.).
Website	Adresse der Mod-Seite.
Header-Bild	Bild, das oben im Installationsprogramm angezeigt wird. Schaltflächen „Durchsuchen...“ und „Löschen“; es wird eine Vorschau angezeigt. Der Pfad wird relativ zum Stammverzeichnis gespeichert.
Beschreibung	Präsentationstext des Mods (mehrzeiliges Feld).

Hinweis – Jede Änderung an einem Feld kennzeichnet das Projekt in der Statusleiste als „Geändert“.

Nexus-Link – Neben dem Auswahlfeld „Spielname“ öffnet eine Schaltfläche „Nexus ↗“ direkt im Browser die Nexus-Mods-Seite des ausgewählten Spiels (auf Basis seines „Slugs“). Diese Schaltfläche ist nur aktiv, wenn das ausgewählte Spiel über einen Slug verfügt.

Die Listen „Spielname“ und „Kategorie“ stammen aus der externen Datei „assets/data/Categories.json“ und können ohne Neukompilierung bearbeitet werden. Jedes Spiel wird dort durch eine Kennung, einen Namen, einen Nexus-„Slug“ und seine Kategorieliste beschrieben:

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "..."]
    }
  }
}
```

4.2 Die Registerkarte „Installationsschritte“

Dies ist das Herzstück des Installationsassistenten. Es basiert auf einer dreistufigen Hierarchie: Schritte → Gruppen → Plugins. Eine Reihe von Registerkarten am oberen Rand stellt die Schritte dar; über die Schaltfläche „+“ wird ein Schritt hinzugefügt (standardmäßig „Schritt N“ genannt). Unterhalb des Schrittnamens (bearbeitbar) und der Schaltfläche „Schritt löschen“ ist der Bildschirm in zwei Spalten unterteilt: links die Gruppen und Plugins, rechts die Details des ausgewählten Plugins.

Gruppen und Auswahltypen

Jede Gruppe hat einen Namen und einen Auswahltyp, der festlegt, wie viele Optionen der Endbenutzer ankreuzen kann. Die fünf verfügbaren Typen sind:

Typ	Bedeutung
SelectExactlyOne	Der Benutzer muss genau eine Option auswählen.
SelectAtMostOne	Höchstens eine Option (oder keine).
SelectAny	Beliebige Anzahl von Optionen (Standardwert).
SelectAll	Alle Optionen sind ausgewählt.
SelectAtLeastOne	Mindestens eine Option.

Die Schaltflächen „Gruppe hinzufügen“ und „Gruppe löschen“ dienen zur Verwaltung der Liste; eine neue Gruppe ist standardmäßig vom Typ „Beliebig auswählen“.

Neuanordnung – Die Anzeigereihenfolge wird angepasst, ohne dass alles neu erstellt werden muss: Die Registerkarten der Schritte enthalten Pfeile „◀“ / „▶“, um den aktuellen Schritt zu verschieben, und jede Gruppe sowie jedes Plugin verfügt über Pfeile „▲“ / „▼“, mit denen es in der Liste um eine Position nach oben oder unten verschoben werden kann.

Plugins und Standardtypen

Ein Plugin stellt eine dem Benutzer angebotene Option dar. Es verfügt über einen Namen, eine Beschreibung, ein Bild (Vorschau, Schaltflächen „Durchsuchen...“ und „Löschen“) sowie einen Standardtyp:

Typ	Bedeutung
Optional	Optionale Option (Standard).
Erforderlich	Option, die immer installiert wird.
Empfohlen	Empfohlene, aber optionale Option.
Nicht verwendbar	Nicht verwendbare Option.
Könnteverwendbar	Möglicherweise nutzbare Option, abhängig von den Bedingungen.

Plugin-Dateien

Der Abschnitt „Quelle“ zeigt in einer vierspaltigen Tabelle (Typ, Quelle, Ziel, Priorität) die vom Plugin installierten Dateien und Ordner an. Die Schaltflächen „Datei hinzufügen“, „Ordner hinzufügen“ und „Entfernen“ dienen zur Verwaltung der Liste. Das Ziel wird automatisch aus der Quelle berechnet: Eine Plugin-Datei (.esp, .esm, .esl, .ba2) wird im Stammverzeichnis unter ihrem Namen abgelegt, während bei

anderen Dateien der Pfad bis zum ersten bekannten Bethesda-Ordner (Texturen, Meshes, Sound, Skripte, Benutzeroberfläche usw.) gekürzt wird. Die Dateien müssen sich im Stammverzeichnis befinden.

Bedingungsflags

Im Abschnitt „Flag-Name:“ kannst du dem Plugin Flags zuweisen (Paare aus Name und Wert). Wenn ein Plugin vom Endbenutzer ausgewählt wird, werden seine Flags gesetzt; sie können dann die Sichtbarkeit eines Schritts, den Typ eines Plugins (Abhängigkeitsmuster) oder die Installation bedingter Dateien beeinflussen. Die Schaltflächen „Flag hinzufügen“ und „Flag entfernen“ dienen zur Verwaltung der Liste.

Automatische Vervollständigung – Die Felder für Name und Wert von Flags und Abhängigkeiten bieten während der Eingabe ein Menü mit Vorschlägen, die aus den bereits im Projekt vorhandenen Elementen stammen. Dadurch werden Tippfehler vermieden, die eine Übereinstimmung unbemerkt zunichte machen würden. Bei einer Dateibedingung schlägt der Wert die Zustände „Aktiv“, „Inaktiv“ oder „Fehlt“ vor. Die Vorschläge sind nicht bindend: Es kann weiterhin frei ein neuer Wert eingegeben werden.

4.3 Die Registerkarte „Erforderliche Installationen“

Auf dieser Registerkarte werden die Dateien und Ordner systematisch aufgelistet, unabhängig von den Auswahlmöglichkeiten des Benutzers. Sie enthält dieselbe vier Spalten umfassende Tabelle (Typ, Quelle, Ziel, Priorität) sowie dieselben Schaltflächen „Datei hinzufügen“, „Ordner hinzufügen“ und „Entfernen“ wie der Dateibereich eines Plugins.

4.4 Die Registerkarte „Bedingte Installationen“

Auf dieser Registerkarte werden Muster gruppiert: Jedes Muster installiert eine Reihe von Dateien nur dann, wenn seine Bedingungen erfüllt sind. Über eine Reihe von Registerkarten mit der Bezeichnung „Muster N“ können Sie ein Muster hinzufügen („+“) oder entfernen („–“).

Für das ausgewählte Muster:

- **Operator** – „Und“ oder „Oder“: Wie die Abhängigkeiten des Musters kombiniert werden.
- **Abhängigkeiten** – eine Liste von Bedingungen vom Typ „Flag“ ([Flag] Name = Wert) oder „Datei“ ([Datei] Name (Status)). Über die Schaltflächen „Abhängigkeit hinzufügen“ und „Abhängigkeit entfernen“ lässt sich die Liste verwalten.
- **Dateien** – Die vier Spalten umfassende Tabelle der Dateien, die installiert werden, wenn die Bedingungen erfüllt sind.

4.5 Das Fenster „Einstellungen“

Zugänglich über „Optionen“ → „Einstellungen“ (Strg+.). Es verfügt über zwei Registerkarten: „Allgemein“ und „Zuletzt verwendete Dateien“. Die Einstellungen werden in der Datei „Config.ini“ gespeichert.

Die Registerkarte „**Allgemein**“ ist in zwei Spalten unterteilt.

Linke Spalte – die Landesflagge und ihr Name:

Element	Funktion
Flag	Ein Klick auf das Bild öffnet die Länderauswahl (siehe unten).
Ländername	Name des Landes in der ausgewählten Sprache (Endonym). Nicht bearbeitbares Feld, wird in zwei Zeilen angezeigt.

Rechte Spalte – die Einstellungen der Benutzeroberfläche:

Einstellung	Beschreibung
Sprache	Sprache der Benutzeroberfläche, wird sofort übernommen. Das Feld bleibt deaktiviert, solange kein Land ausgewählt ist; anschließend werden die in diesem Land gesprochenen Sprachen angeboten.
Design	Dunkel, Hell oder System.
Schriftgröße	Größe des Texts in der Benutzeroberfläche.
Zeilenumbrüche verarbeiten	Interpretiert \n-Sequenzen in Beschreibungen.
Max. Anzahl der zuletzt verwendeten Dateien	Anzahl der gespeicherten Einträge.
Fensterbreite/-höhe	Gespeicherte Abmessungen des Fensters.

Länderauswahl – durch Klicken auf die Flagge öffnet sich ein Fenster, in dem die 244 Flaggen als Raster mit vertikalem und horizontalem Bildlauf angezeigt werden. Über das Feld „Filter:“ können Sie die Liste nach französischem Namen, englischem Namen oder Ländercode eingrenzen. Der Ländername erscheint unter jeder Flagge.

Die Auswahl des Landes bestimmt die Liste der angebotenen Sprachen: XIMOD stützt sich auf die Datei „CountryLanguages.json“, in der alle in den einzelnen Ländern gesprochenen Sprachen aufgeführt sind – bis zu 844 für Papua-Neuguinea, daher das Scrollen in der Dropdown-Liste.

Das Flaggenraster lässt sich auch **über die Tastatur** bedienen: Die Pfeiltasten (oder der Ziffernblock 8/2/4/6) bewegen den Auswahlrahmen, Bild auf/Bild ab blättern um einen Bildschirm, Pos1/Ende springen zur ersten und letzten Flagge, Tab und Umschalt+Tab wechseln um eine Flagge vor und zurück, Eingabe bestätigt das markierte Land und Esc schließt das Fenster. Ein Löschesymbol (rotes Kreuz) rechts vom Feld „Filter:“ leert es mit einem Klick, und das Fenster öffnet sich zentriert auf dem Hauptfenster (siehe § 2.6).

4.6 Skripte vor und nach dem Speichern

XIMOD Architect kann beim Speichern automatisch ein Skript ausführen: ein Skript vor dem Speichern, das unmittelbar vor dem Schreiben der FOMOD-Dateien gestartet wird, und ein Skript nach dem Speichern, das unmittelbar danach gestartet wird. Sie dienen dazu, die Aufgaben rund um die Mod-Erstellung zu automatisieren – zum Beispiel das Bereinigen alter Archive vor dem Speichern, das Erhöhen einer Build-Nummer oder das automatische Verpacken der Mod in ein veröffentlichungsfertiges Archiv, sobald die FOMOD-Dateien geschrieben sind.

Sie werden über „Optionen → Skript vor dem Speichern...“ und „Optionen → Skript nach dem Speichern...“ bearbeitet. Das Fenster zeigt einen Hilfetext, die Liste der verfügbaren Makros, einen Eingabebereich mit fester Zeichenbreite sowie die Schaltflächen „Speichern“ und „Abbrechen“. Der Inhalt beider Skripte wird in der Datei „Config.ini“ (Schlüssel „PreSaveScript“ und „PostSaveScript“) gespeichert und daher von einer Sitzung zur nächsten beibehalten.

So funktioniert es: Beim Speichern ersetzt XIMOD zunächst die Makros durch ihre Werte, schreibt das Ergebnis in eine temporäre Datei und führt es anschließend mit dem Systeminterpreter aus – unter Windows ist dies eine von cmd gestartete .bat-Befehlsdatei, unter Linux und macOS ein von sh gestartetes .sh-Shell-Skript. Die temporäre Datei wird anschließend gelöscht. Das Pre-Save-Skript wird vor dem Schreiben der XML-Dateien ausgeführt, das Post-Save-Skript unmittelbar danach; ein leer gelassenes Skript wird einfach ignoriert.

Vor der Ausführung werden die folgenden Makros im Skripttext ersetzt:

Makro	Ersetzter Wert
\$MODNAME\$	Mod-Name.
\$MODAUTHOR\$	Name des Autors.
\$MODVERSION\$	Mod-Version.
\$MODROOT\$	Pfad zum Stammverzeichnis.
\$DATE\$	Aktuelles Datum (JJJJ-MM-TT).
\$TIME\$	Aktuelle Uhrzeit (HH:MM:SS).
\$RANDOM\$	Zufallszahl.

Beispiel – Skript vor dem Speichern (Windows)

Lösche die bereits im Stammverzeichnis des Mods vorhandenen .rar-Archive, um vor dem Speichern mit einem leeren Ordner zu beginnen:

```
del "$MODROOT$\*.rar" /q
```

Beispiel – Skript nach dem Speichern (Windows)

Sobald die FOMOD-Datei geschrieben wurde, verpacke den gesamten Mod-Inhalt automatisch mit WinRAR in ein .rar-Archiv, das nach dem Mod-Namen und der Version benannt ist:

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

Zeile für Zeile: „d:“ wechselt zum Laufwerk D:, „cd \"\$MODROOT\$““ wechselt in das Stammverzeichnis des Mods, anschließend erstellt WinRAR das Archiv (-r rekursiv, -ep1, um die übergeordnete Verzeichnisstruktur nicht zu kopieren) mit dem Namen beispielsweise „Dawnlight_1.2.0.rar“.

Entsprechung unter Linux/macOS

Unter Linux und macOS wird das Skript als Shell-Skript interpretiert. Die gleiche Paketierung nach dem Speichern lässt sich beispielsweise mit zip schreiben:

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

Hinweis – Diese Skripte führen echte Systembefehle mit Ihren Benutzerrechten aus; ein Befehl wie „del“ oder „rm“ ist destruktiv. Testen Sie sie in einem Testprojekt, bevor Sie sie für eine echte Mod verwenden.

4.7 Das Fenster „Über“

Dieses Fenster wird über „Hilfe“ → „Über“ (F1) geöffnet und zeigt den Namen der Anwendung, ihre Versionsnummer, eine kurze Beschreibung („Ein plattformübergreifendes Tool zum Erstellen von FOMOD-Installationsprogrammen für Bethesda-Spiel-Mods.“) sowie den Hinweis „Lizenziert unter MIT“ an. Über die Schaltfläche „OK“ wird es geschlossen.

4.8 Erstellen der FOMOD-Dateien

Beim Speichern wandelt XIMOD Architect das Projekt in zwei standardisierte XML-Dateien um, die im Unterordner „fomod“ im UTF-8-Format gespeichert werden.

Vor dem Speichern wird das Projekt überprüft: Enthält es Unregelmäßigkeiten (fehlender Mod-Name, fehlender Schritt oder erforderliche Datei, ein Schritt oder eine Gruppe ohne Namen, eine Gruppe ohne Plugin), listet ein Fenster diese alle auf und bietet die Option „Trotzdem speichern“ an – das Speichern wird niemals blockiert, sodass Sie ein noch in Arbeit befindliches Projekt bewusst speichern können.

Schema-Validierung – Zusätzlich zu dieser Projektprüfung erfolgt eine strukturelle Überprüfung der generierten „ModuleConfig.xml“ anhand des in Rust geschriebenen ModConfig 5.0-Schemas (ohne externe XSD-Engine): Elementhierarchie und Kardinalität, obligatorische Attribute und Aufzählungswerte (Auswahl- und Plugin-Typen, Dateistatus, Operatoren). Jede Anomalie wird anhand ihrer Zeile und Spalte lokalisiert. Der Zugriff erfolgt auf drei Arten: über den Befehl „Optionen → FOMOD validieren“, der einen vollständigen Bericht anzeigt; live im XML-Editor, sobald das Dokument wohlgeformt ist; sowie integriert in das Fenster zur Speicherbestätigung.

info.xml enthält die Metadaten unter einem <fomod>-Stammelement: den Namen (Name), den Autor (Author), die Version (Version), die Website (Website), die Beschreibung (Description), die Kategorie (Groups) und das Spiel (Game). Das Game-Element ist eine XIMOD-spezifische Erweiterung, die von Mod-Managern harmlos ignoriert wird und dazu dient, beim Neuladen die korrekte Liste der Kategorien wiederherzustellen.

ModuleConfig.xml beschreibt die Installation unter einem <config>-Stammelement: den Modulnamen, das Header-Bild, die erforderlichen Dateien (requiredInstallFiles), die Schritte (installSteps → group → plugin) und die bedingten Installationen (conditionalFileInstalls). Jedes Plugin beschreibt seine Beschreibung, sein Bild, seine Bedingungsflags, seine Dateien und seinen Typ. Abhängigkeiten werden durch einen „And“- oder „Or“-Operator kombiniert.

4.9 Übersetzungseditor

Dieses Tool ist über „Optionen → Übersetzung“ zugänglich und ermöglicht es Ihnen, die Benutzeroberfläche zu übersetzen, ohne die Dateien manuell bearbeiten zu müssen, sowie eine bestehende Übersetzung zu korrigieren.

Fensterkopf

Feld	Rolle
Flag	Auswahl des Landes, die die Liste der übersetzbaren Sprachen bestimmt.
Endonym des Landes	Name des Landes in der übersetzten Sprache, dargestellt mit der gewählten Schriftart.
Endonym der Sprache	Name der Sprache in dieser Sprache selbst (z. B. „Français“).
Autor	Name des Übersetzers.
Angezeigte Sprache	In Spalte 2 angezeigte Referenzsprache (unter den Sprachen, für die bereits eine main.ftl-Datei vorhanden ist).
Zu übersetzende Sprache	Zielsprache, ausgewählt aus den Sprachen des Landes.

Feld	Rolle
Schriftart	Für diese Sprache verwendete Schriftart. Über die Schaltfläche „Durchsuchen...“ wird ein Dateiauswahlfenster geöffnet, das auf den Ordner „assets/fonts“ verweist.
Google Fonts	Öffnet die Website fonts.google.com im Browser.

Wichtige Einschränkung: Die Schriftart muss im Ordner „assets/fonts“ installiert sein. Eine an einem anderen Ort befindliche Schriftart wird abgelehnt, da das Programm sie über einen relativen Pfad zu diesem Ordner lädt und sie beim nächsten Start nicht finden würde. Falls die Schriftart fehlt, müssen Sie sie daher von Google Fonts herunterladen und dorthin kopieren, bevor Sie sie auswählen.

Ebenso kann nur eine Sprache übersetzt werden, die über einen **ISO 639-3-Code** verfügt: Dieser Code benennt den Übersetzungsordner und verknüpft die Sprache mit ihrer Schriftart und ihren Referenzdaten. Andernfalls sind die Speichern-Schaltflächen deaktiviert.

Übersetzungstabelle – drei Spalten:

Spalte 1	Spalte 2	Spalte 3
Schlüssel (nicht bearbeitbar)	Bezeichnung in der angezeigten Sprache	Eingabefeld für die Übersetzung

Token-Schutz: Elemente, die nicht übersetzt werden dürfen (Makros wie \$MODNAME\$, Variablen wie { \$num }), erscheinen niemals unverändert in Spalte 2 und werden automatisch beibehalten.

- Token am Anfang oder Ende des Textes: wird ausgeblendet und beim Speichern wieder eingefügt.
- Token in der Mitte des Textes: wird durch eine kompakte Markierung [[1]], [[2]]... ersetzt, um es beizubehalten und neu zu positionieren.

Der Anwendungsname (Schlüssel „app-title“) ist bewusst ausgeschlossen und nicht übersetzbar.

Speichern – Die Schaltfläche „Speichern“ schreibt die Datei „assets/locales/<Sprache>/main.ftl“, der ein Metadatenblock in Form von Fluent-Kommentaren vorangestellt wird (die Datei bleibt somit vollständig gültig):

```
# XIMOD Architect – Übersetzungsmetadaten
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

Der Header beschreibt ausschließlich die **Sprache**: Er enthält keine Länderangaben, da dieselbe Sprache in mehreren Ländern gesprochen wird (Deutsch beispielsweise sowohl in Deutschland als auch in der Schweiz). Das Land wird durch die gewählte Flagge bestimmt, und XIMOD übernimmt die Zuordnung von Ländern zu Sprachen anhand der Dateien „Countries.json“ und „CountryLanguages.json“.

Ausfüllen der Endonym-Felder – sobald das Land (Flagge) und die Sprache ausgewählt sind, füllt XIMOD die beiden Felder aus:

- **Endonym des Landes:** wird für das ausgewählte Land immer aus der Datei „Countries.json“ ermittelt; es folgt daher der Flagge (im Deutschen wird „Deutschland“ für Deutschland und „Schweiz“ für die Schweiz angezeigt).

- **Endonym der Sprache:** wird aus dem Datei-Header (# @langname) übernommen, sofern vorhanden; andernfalls – bei einer alten Datei ohne Header oder einer noch nicht übersetzten Sprache – wird der Wert aus „Languages.json“ übernommen.

Die beiden Felder bleiben bearbeitbar: Der Benutzer kann dort Korrekturen vornehmen, die beim Speichern in die Referenzdateien übernommen werden.

Durch das Speichern werden die **Referenzdaten** aktualisiert: das Endonym und die Schriftart der Sprache in „Languages.json“ sowie das Länderendonym in „Countries.json“ (für das Länder-/Sprachpaar). Falls die Sprache für dieses Land dort noch nicht aufgeführt war, wird ein Eintrag angelegt.

Da die Kopfzeile das Land nicht mehr enthält, schreibt XIMOD neben der Datei „main.ftl“ eine kleine, durch Tabulatoren getrennte Datei **namens „countryEndonyms.tsv“**, in der das Ländername für die jeweilige Sprache erfasst wird – eine Zeile pro Land in der Form „Ländercode, Sprachcode, Ländername“. Diese Datei ermöglicht es, den Ländernamen wiederherzustellen, wenn nur der Sprachordner verfügbar ist.

Versenden der Übersetzung – Die Schaltfläche „Senden...“ erstellt ein ZIP-Archiv, das die Übersetzungsdatei (main.ftl, im entsprechenden Sprachordner) und, falls vorhanden, die zugehörige Datei countryEndonyms.tsv enthält; XIMOD öffnet den Archivordner und bereitet anschließend eine vorausgefüllte E-Mail vor. Die sprachspezifischen Informationen (Sprache, Schriftart, Sprachbezeichnung, Autor) werden im Header der Datei main.ftl übermittelt; der Ländername wird in der Datei „countryEndonyms.tsv“ übermittelt. Weder „Languages.json“ noch „Countries.json“ werden angehängt – es werden nur die tatsächlich verwendeten Codes übertragen, was den Nutzungsbedingungen der ISO 639-3-Codes entspricht. Der Nutzer hängt die Datei selbst an und versendet sie: Ohne eine explizite Aktion verlassen keine Daten den Rechner.

Hinweis – Senden Sie das Paket erst, wenn die Übersetzung vollständig ist: Jeder nicht übersetzte Schlüssel wird mit einem leeren Wert gespeichert, der dann in der Benutzeroberfläche ohne Text angezeigt würde.

Tastaturnavigation — die Tabelle lässt sich ohne Maus bedienen (siehe Anhang B): Die Tasten Auf/Ab und Bild auf/Bild ab blättern durch die Zeilen, Pos1/Ende springen zur ersten und letzten, und Eingabe setzt den Cursor in das Eingabefeld der markierten Zeile. Während der Eingabe wechselt Tab zum nächsten Feld und Umschalt+Tab zum vorherigen, sodass Übersetzungen nacheinander eingegeben werden können, ohne die Tastatur zu verlassen. Die Anzahl der von Bild auf/Bild ab bewegten Zeilen passt sich der Fensterhöhe an.

4.10 XML-Editor

Dieses Tool für fortgeschrittene Benutzer ist über „Optionen“ → „XML-Editor“ → „info.xml“ oder „ModuleConfig.xml“ zugänglich und ermöglicht es Ihnen, das generierte XML anzuzeigen und direkt zu bearbeiten.

Es öffnet sich in einem **eigenständigen Fenster**, das frei verschoben und in der Größe angepasst werden kann, auch auf einem zweiten Bildschirm.

Zwei Modi

- **Schreibgeschützt** (Standard): Das XML wird mit Syntaxhervorhebung und hängender Einrückung angezeigt – zu lange Zeilen werden umgebrochen und unterhalb des Zeilenanfangs ausgerichtet, ähnlich wie bei Notepad++. Das Hauptfenster bleibt weiterhin nutzbar.
- **Bearbeiten** (Schaltfläche „Ändern“): Der Text wird bearbeitbar und die Registerkarten des Hauptfensters werden gesperrt, sodass nur eine einzige Quelle maßgeblich ist.

Syntaxhervorhebung: Tags, Attributnamen, in Anführungszeichen gesetzte Werte, Kommentare, Deklarationen und Text werden durch Farben unterschieden, die zum hellen oder dunklen Design passen.

Zeilennummerierung: In einer Seitenleiste werden die Nummern angezeigt; die Fortsetzungszeilen eines Zeilenumbruchs bleiben leer.

Live-Validierung: Während der Eingabe wird das XML kontinuierlich überprüft. Eine grüne Anzeige bestätigt ein wohlgeformtes Dokument; im Fehlerfall gibt eine rote Meldung die Zeile und Spalte an, und das fehlerhafte Token wird unterstrichen.

Änderungen übernehmen: Die Schaltfläche „Übernehmen“ analysiert den Text erneut und aktualisiert das Projekt. Im Fehlerfall wird die Änderung abgelehnt und der Text bleibt bearbeitbar. Die Schaltfläche „Abbrechen“ generiert den Text aus dem Projekt neu.

4.11 Schriftartenverwaltung

XIMOD Architect wird mit einer Reihe von Noto-Schriftarten ausgeliefert, die alle Schriftsysteme der aufgeführten Sprachen abdecken und im Verzeichnis „assets/fonts“ abgelegt sind.

Diese Schriftarten werden **dynamisch** geladen: Zu jedem Zeitpunkt werden nur die tatsächlich benötigten installiert, nämlich die Schriftart der Benutzeroberflächensprache und die der Sprachen des ausgewählten Landes. Der Speicherbedarf bleibt gering – für die meisten Länder reicht eine einzige Schriftart aus.

Dieser Mechanismus gewährleistet die korrekte Darstellung nicht-lateinischer Schriften (Thai, Burmesisch, Amharisch, Hmong usw.), die andernfalls als leere Quadrate erscheinen würden.

Die jeder Sprache zugeordnete Schriftart ist in „Languages.json“ angegeben und kann im Übersetzungseditor geändert werden.

Dieses dynamische Laden gilt auch für das Fenster „Eigenschaften“ und den Übersetzungseditor: Die Schriftarten aller dort angezeigten Sprachen werden ebenfalls geladen, damit kein Name und keine Beschriftung als leere Kästchen erscheint.

XIMOD Architect wird mit **32 Oberflächenübersetzungen** ausgeliefert, die jeweils den gesamten Satz an Zeichenketten abdecken, einschließlich nichtlateinischer Schriften: Japanisch (jpn), vereinfachtes Chinesisch (zho), Koreanisch (kor) und Russisch (rus). Englisch (eng) und Französisch (fra) sind die manuell gepflegten Referenzübersetzungen; die übrigen stammen aus Beiträgen, die von Muttersprachlern über den Übersetzungseditor überprüft werden können.

4.12 Zwei FOMODs zusammenführen

Mit dem Befehl „Datei → FOMOD zusammenführen...“ können Sie zwei Installationsprogramme kombinieren, indem Sie deren Installationsinhalte zusammenführen. Das aktuell geöffnete Projekt fungiert als Empfänger; es muss daher zuvor erstellt oder geöffnet worden sein, andernfalls bleibt der Befehl deaktiviert.

Ein Klick öffnet einen Dateiauswähler: Wählen Sie die Datei „ModuleConfig.xml“ des zu integrierenden FOMODs (des Spenders) aus. Dessen Schritte, erforderlichen Dateien und bedingten Installationen werden dann an das Ende derjenigen des Empfängerprojekts angehängt. Die Metadaten des Empfängerprojekts – Name, Autor, Version, Kategorie, Header-Bild – bleiben erhalten: Es wird nur der Installationsinhalt importiert.

Nach dem Zusammenführen wird das Projekt als geändert markiert: Denken Sie daran, es zu speichern (Strg+S), um das Ergebnis in die FOMOD-Dateien zu schreiben. Der Vorgang kann wiederholt werden, um mehrere FOMODs nacheinander zu integrieren.

Hinweis – Es wird nur die Datei „ModuleConfig.xml“ des Spenderprojekts gelesen; dessen eigene Metadaten (info.xml) werden nicht importiert. Überprüfen Sie nach dem Zusammenführen, ob die Prioritäten und Bedingungen über die kombinierten Schritte hinweg konsistent bleiben.

4.13 Exportieren des Distributionsarchivs

Der Befehl „Datei → Distributionsarchiv exportieren...“ erstellt in einem einzigen Vorgang das Archiv, das zum Hochladen (auf Nexus Mods oder einen anderen Mod-Manager) bereit ist. XIMOD schreibt zunächst die FOMOD-Dateien (fomod/info.xml und fomod/ModuleConfig.xml) in das Stammverzeichnis und komprimiert anschließend den gesamten Ordner im ZIP-Format. Der Befehl ist deaktiviert, solange kein Stammverzeichnis definiert ist.

Ein Auswahlfeld bietet einen Standardnamen in der Form „<name>-<version>.zip“ an (bereinigt von unzulässigen Zeichen); Sie können diesen vor dem Speichern ändern. Die Statusleiste zeigt dann die Anzahl der geschriebenen Dateien und den Pfad des Archivs an.

Archivstruktur – Der Unterordner „fomod/“ und die Mod-Dateien befinden sich im Stammverzeichnis des Archivs, genau wie es Mod-Manager erwarten. Interne Pfade verwenden den Schrägstrich „/“ (plattformübergreifende Kompatibilität).

Hinweis – Unnötige Dateien (.git, .DS_Store, Thumbs.db, desktop.ini...) und das Archiv selbst werden automatisch ausgeschlossen. Stellen Sie daher vor dem Export sicher, dass sich alle Mod-Dateien tatsächlich im Stammverzeichnis befinden.

4.14 Das Fenster „Eigenschaften“

Dieses eigenständige Fenster, das über „Optionen → Eigenschaften...“ geöffnet wird – verschiebbar und in der Größe anpassbar, auch auf einem zweiten Bildschirm –, bietet einen schreibgeschützten Explorer der eingebetteten Länder-/Sprachdatenbank. Die Datenbank kann hier nicht geändert werden; es handelt sich um ein Nachschlagewerk.

Registerkarte „Länder“: Für ein ausgewähltes Land zeigt das Fenster dessen Flagge, dessen Namen auf Französisch und Englisch, dessen Amtssprachen (mit Endonym, Schriftart und ISO-639-3-/639-1-Codes) sowie die Liste aller dort gesprochenen Sprachen an.

Registerkarte „Sprachen“: Für eine ausgewählte Sprache zeigt das Fenster deren ISO-Codes und Schriftart an und führt eine Rückwärtssuche nach den Ländern durch, in denen diese Sprache gesprochen wird.

Tastaturnavigation — sowohl die Länder- als auch die Sprachliste lassen sich über die Tastatur bedienen (siehe Anhang B): Auf/Ab, Bild auf/Bild ab (deren Schrittweite sich der Fensterhöhe anpasst), Pos1/Ende und Eingabe zur Auswahl des markierten Eintrags. Tab und Umschalt+Tab bewegen die Auswahl innerhalb der Liste, ohne je auf der Flagge zu landen. Jedes Feld „Filter:“ besitzt rechts ein Löschesymbol (rotes Kreuz), und das Fenster schließt sich mit Esc.

4.15 Befehlszeilenmodus (CLI)

Wird XIMOD Architect mit Argumenten gestartet, öffnet es kein Fenster: Es führt den angeforderten Befehl aus und beendet sich anschließend mit einem Exit-Code (0 bei Erfolg, ungleich Null bei Fehler oder

Validierungsanomalie). Dieser Modus ermöglicht es, das Neuerstellen und Verpacken eines oder mehrerer FOMODs in einem Skript oder einer Continuous-Integration-Pipeline zu automatisieren.

Hinweis – Unter Windows wird die ausführbare Datei als „Windows“-Anwendung kompiliert und bindet sich daher über „AttachConsole“ an die Konsole, von der aus sie gestartet wurde, um dort ihre Meldungen auszugeben. Starten Sie das Programm über eine Eingabeaufforderung (cmd oder PowerShell), um die Ausgabe zu sehen; ein Doppelklick auf einen CLI-Befehl würde nichts anzeigen.

Die verfügbaren Befehle sind:

Befehl	Wirkung
ximod-architect validate <root>	Überprüft das FOMOD des Verzeichnisses (Projektprüfungen und Konformität mit dem ModConfig 5.0-Schema); jede Anomalie wird lokalisiert. Der Exit-Code ist ungleich Null, wenn eine Anomalie gefunden wird.
ximod-architect build <root>	Schreibt die Dateien „fomod/info.xml“ und „fomod/ModuleConfig.xml“ des vorhandenen Projekts (neu).
ximod-architect package <root> [-o FILE]	Schreibt das FOMOD und erstellt anschließend das .zip-Verteilungsarchiv. Mit der Option -o können Sie den Ausgabepfad angeben; andernfalls wird das Archiv im aktuellen Verzeichnis erstellt.
ximod-architect help	Zeigt die Hilfe (Liste der Befehle) an.
ximod-architect version	Zeigt die Versionsnummer an.

Dabei bezeichnet <root> das Stammverzeichnis des Mods (den Ordner, der den Unterordner „fomod“ enthält – oder enthalten wird).

5. FOMOD-Tag-Referenz

Dieser Abschnitt beschreibt die Tags, die tatsächlich von XIMOD Architect erzeugt werden. Die beiden Dateien werden in UTF-8 (mit einer Byte-Order-Mark) im „fomod“-Unterordner des Mods geschrieben: „info.xml“ wird in Schritten von vier Leerzeichen eingerückt, „ModuleConfig.xml“ mit einem Tabulator. Beim Lesen wird die Kodierung dank der Byte-Order-Mark automatisch erkannt, wodurch es möglich ist, sowohl UTF-8- als auch UTF-16-Dateien (LE oder BE) zu öffnen, die von anderen Tools erstellt wurden.

5.1 Die Datei „info.xml“

Die Metadaten befinden sich unter einem <fomod>-Stammelement. Die Elemente werden in der Reihenfolge der folgenden Tabelle geschrieben; diejenigen, die mit „falls nicht leer“ gekennzeichnet sind, werden weggelassen, wenn das entsprechende Feld leer bleibt.

Tag	Inhalt	Geschrieben
<Name>	Mod-Name.	Immer
<Author>	Autor.	Falls nicht leer
<Version>	Version.	Wenn nicht leer
<Website>	URL der Mod-Seite.	Falls nicht leer
<Description>	Beschreibung.	Wenn nicht leer
<Groups>	Kategorie des Mods.	Immer
<Game>	Kennung des ausgewählten Spiels. XIMOD-spezifische Erweiterung (außerhalb der FOMOD-Spezifikation), die von Mod-Managern ignoriert wird.	Wenn nicht leer

Vollständiges Beispiel:

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
  <Name>My Awesome Mod</Name>
  <Author>Patrice</Author>
  <Version>1.0.0</Version>
  <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
  <Description>A sample mod for the manual.</Description>
  <Groups>Player homes</Groups>
  <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 Die Datei „ModuleConfig.xml“

Die Installationslogik unter einem <config>-Stammelement, das auf das Schema „ModConfig5.0.xsd“ verweist. Die Blöcke erscheinen in folgender Reihenfolge: Modulname, Header-Bild, erforderliche Dateien, Installationsschritte, bedingte Installationen.

Stamm und Kopfzeile

Tag / Attribut	Rolle
<config>	Stammelement. Enthält die Attribute „xmlns:xsi“ und

Tag / Attribut	Rolle
	„xsi:noNamespaceSchemaLocation“ (Schema „ModConfig5.0.xsd“).
<moduleName>	Modulname, der oben im Assistenten angezeigt wird.
<moduleImage path="..." />	Header-Bild (leeres Tag, wird nur geschrieben, wenn ein Bild definiert ist).

Dateien und Ordner

Tag / Attribut	Rolle
<file source="..." destination="..." priority="..." />	Kopie einer Datei. source = Pfad im Archiv (relativ zum Stammverzeichnis); destination = Zielort unter „Data“; priority = Reihenfolge bei der Installation.
<folder source="..." destination="..." priority="..." />	Kopie eines gesamten Ordners (gleiche Attribute).
<requiredInstallFiles>	Container der systematisch installierten Dateien.
<files>	Container der Dateien, die mit einer Option oder einem bedingten Muster verknüpft sind.

Schritte und Gruppen

Tag / Attribut	Rolle
<installSteps order="Explicit">	Container für die Schritte (die festgelegte Reihenfolge bleibt erhalten).
<installStep name="...">	Eine Seite des Assistenten.
<visible>	Sichtbarkeitsbedingungen des Schritts (enthält <dependencies>).
<optionalFileGroups>	Container für die Optionsgruppen des Schritts.
<group name="..." type="...">	Eine Gruppe; type = Auswahlregel (siehe unten).
<plugins order="Explicit">	Behälter für die Optionen der Gruppe.

Werte des Attributs „type“ eines <group>-Elements:

type	Einschränkung
SelectExactlyOne	Genau eine Option.
SelectAtMostOne	Höchstens eine Option.
SelectAny	Beliebige Anzahl.
SelectAll	Alle, obligatorisch.
SelectAtLeastOne	Mindestens eines.

Optionen (Plugins)

Tag / Attribut	Rolle
<plugin name="...">	Eine Option, die dem Benutzer angeboten wird.
<description>	Beschreibender Text zur Option (wird immer angegeben).
<image path="..." />	Bild zur Veranschaulichung der Option (falls definiert).
<conditionFlags>	Container für die Flags, die gesetzt werden, wenn die Option ausgewählt wird.
<flag name="...">value</flag>	Ein Flag: Name im Attribut, Wert im Inhalt.
<typeDescriptor>	Beschreibt den Typ der Option (einfach oder bedingt).
<type name="..." />	Einfacher Typ (wenn kein Abhängigkeitsmuster definiert ist).
<dependencyType>	Dynamischer Typ (wenn Muster vorhanden sind).
<defaultType name="..." />	Standardtyp, der angewendet wird, wenn kein Muster zutrifft.

Werte von <type> / <defaultType>:

Typ	Auswirkung im Assistenten
Optional	Option nicht markiert, frei aktivierbar.
Erforderlich	Obligatorische Option, kann nicht deaktiviert werden.
Empfohlen	Option ist standardmäßig aktiviert, kann deaktiviert werden.
Nicht verwendbar	Option ist deaktiviert (kann nicht ausgewählt werden).
Möglicherweise verwendbar	Verwendbar, wird jedoch als potenziell risikobehaftet gekennzeichnet.

Abhängigkeiten und Bedingungen

Tag / Attribut	Rolle
<dependencies operator="...">	Gruppe von Bedingungen; operator = And oder Or.
<fileDependency file="..." state="..." />	Bedingung für eine Datei; state = „Active“, „Inactive“ oder „Missing“.
<flagDependency flag="..." value="..." />	Bedingung für ein Flag und dessen erwarteten Wert.
<conditionalFileInstalls>	Abschnitt zu den bedingten Installationen.
<patterns> / <pattern>	Liste der Muster; jedes Muster ordnet <dependencies> den <files> zu (oder, innerhalb eines typeDescriptors, einem <type>).

6. Die Datei „ModuleConfig.xml“ im Detail

In Kapitel 5 wurden die Tags als Referenztabelle vorgestellt. In diesem Kapitel werden sie nacheinander in verständlicher Sprache durchgenommen, um zu erklären, was jedes einzelne bewirkt, welche Optionen es akzeptiert und vor allem, welche konkreten Auswirkungen diese Optionen auf den Installationsassistenten haben, den der Nutzer des Mods zu sehen bekommt. Vorkenntnisse in XML sind nicht erforderlich: XIMOD Architect erstellt diese Datei für Sie; sie zu verstehen hilft Ihnen lediglich dabei, zuverlässigere Installationsprogramme zu entwerfen.

Ein paar Begriffe zur Erklärung. Eine XML-Datei besteht aus „Tags“, die in spitzen Klammern stehen. Ein öffnendes Tag `<step>` gehört zu einem schließenden Tag `</step>`; alles, was zwischen diesen beiden steht, gehört zu diesem Tag. Ein Tag kann „Attribute“ enthalten, d. h. Einstellungen in der Form `name="value"`. Somit ist `<group name="Resolution" type="SelectExactlyOne">` ein „group“-Tag mit zwei Attributen. Schließlich schließt sich ein Tag ohne Inhalt in sich selbst, wie z. B. `<file ... />` (beachte das abschließende `/`).

6.1 Wie der Installer die Datei liest

Wenn der Nutzer deinen Mod installiert, liest sein Manager (Vortex, MO2, NMM) die Datei „ModuleConfig.xml“ von oben nach unten und erstellt einen Assistenten: eine Reihe von Seiten (die Schritte), auf denen jeweils Gruppen von Optionen zum Ankreuzen angeboten werden. Am Ende sammelt der Manager alle Dateien, die den ausgewählten Optionen entsprechen, und kopiert sie in das Spiel. Drei Punkte, die du beachten solltest:

- **Die Reihenfolge ist entscheidend:** Die Schritte werden in der Reihenfolge angezeigt, in der sie in der Datei erscheinen;
- **Das Aktivieren einer Option führt nicht sofort zu einer Installation:** Durch das Aktivieren einer Option wird lediglich eine Auswahl gespeichert; das eigentliche Kopieren der Dateien erfolgt erst ganz am Ende;
- **Bedingungen können alles beeinflussen:** Einen Schritt ausblenden, den Status einer Option ändern oder die Installation zusätzlicher Dateien auslösen – das ist die Aufgabe von Flags und Abhängigkeiten (§ 6.6).

6.2 Die Wurzel und der Header

`<config>` – der Stamm. Die gesamte Konfiguration passt in ein einziges `<config>...</config>`-Tag. Es enthält zwei technische Attribute (`xmlns:xsi` und `xsi:noNamespaceSchemaLocation`), die das Validierungsschema (ModConfig 5.0) festlegen. Sie müssen diese niemals ändern: XIMOD schreibt sie immer identisch.

`<moduleName>` – der Mod-Name, der oben im Assistenten angezeigt wird. Es handelt sich um den Namen, der auf der Registerkarte „Mod-Info“ eingegeben wurde.

`<moduleImage path="..." />` – das Bannerbild, das oben im Assistenten angezeigt wird. Das Attribut „path“ gibt den Speicherort relativ zum Stammverzeichnis des Mods an (z. B. `fomod\header.png`). Das Tag ist leer (es schließt sich selbst). Ohne Bild fehlt es einfach.

6.3 Dateien installieren

Zwei Tags beschreiben, was in das Spiel kopiert wird: `<file>` kopiert eine einzelne Datei, `<folder>` kopiert einen ganzen Ordner mit seinem gesamten Inhalt. Beide akzeptieren dieselben drei Attribute:

Attribut	Funktion	Beispiel
source	Pfad des Elements im Archiv deiner Mod, relativ zum Stammverzeichnis.	tex\2k
destination	Ort, an dem das Element abgelegt werden soll, relativ zum Datenordner des Spiels.	textures
priority	Installationsreihenfolge im Konfliktfall (siehe unten).	0

Das Attribut „priority“ und seine Auswirkungen. Es handelt sich um eine Zahl, die die Kopierreihenfolge festlegt, wenn zwei Optionen eine Datei am selben Ort installieren. Die Regel lautet: Optionen mit niedriger Priorität werden zuerst installiert, solche mit hoher Priorität danach – und da die zuletzt geschriebene Option Vorrang hat, „überschreibt“ eine höhere Priorität eine niedrigere. Beispiel: Wenn eine Basisoption eine Textur mit der Priorität „0“ installiert und eine Option „hohe Auflösung“ dieselbe Textur mit der Priorität „1“ installiert, bleibt die Version mit hoher Auflösung erhalten, unabhängig von der Anzeigereihenfolge der Optionen. Belassen Sie den Wert überall auf 0, solange Sie keinen absichtlichen Konflikt zu schlichten haben.

Hinweis – Das Ziel wird von XIMOD automatisch anhand der Quelle berechnet: Es erkennt die Bethesda-Ordner (Textures, Meshes, Sounds, Skripte...). Normalerweise musst du es nicht manuell eingeben.

Diese Tags tauchen an drei Stellen auf: in `<requiredInstallFiles>` (Dateien, die immer installiert werden), in den `<files>` einer Option (werden installiert, wenn die Option ausgewählt wird) und in den `<files>` eines bedingten Musters (werden installiert, wenn die Bedingungen erfüllt sind).

6.4 Schritte, Gruppen und Optionen

Dies ist das Herzstück des Assistenten. Die Hierarchie ist einfach: Ein Schritt enthält Gruppen, eine Gruppe enthält Optionen.

`<installSteps order="Explicit">` fasst alle Schritte zusammen; `order="Explicit"` bedeutet „die von mir festgelegte Reihenfolge einhalten“. Jede Seite des Assistenten ist ein `<installStep name="...">`, dessen `name`-Attribut den oben auf der Seite angezeigten Titel angibt.

`<visible>` – den Schritt bedingt anzeigen. Ein Schritt kann nur angezeigt werden, wenn bestimmte Bedingungen erfüllt sind; diese werden in einem `<visible>`-Tag festgelegt (siehe § 6.6). Ohne `<visible>` wird der Schritt immer angezeigt. Beispiel: Einen Schritt „Erweiterte Einstellungen“ nur anzeigen, wenn der Benutzer zuvor „Experteninstallation“ ausgewählt hat.

`<optionalFileGroups>` enthält die Gruppen des Schritts. Jedes `<group name="..." type="...">` hat einen Namen (Abschnittstitel) und vor allem einen Typ, der die Auswahlregel festlegt – also, was der Benutzer ankreuzen darf:

Typ	Was der Benutzer sieht und tun kann
„Wähle genau eine Option aus“	Optionsfelder: Der Benutzer muss eine Option auswählen, und zwar genau eine. Weder null noch zwei. Ideal für exklusive Varianten (eine einzige Texturauflösung).
SelectAtMostOne	Wie oben, aber der Nutzer kann auch keine Option auswählen: „Entweder eine Option oder gar keine“.
SelectAny	Unabhängige Kontrollkästchen: Der Benutzer kann beliebige Kästchen aktivieren, von keinem bis zu allen. Der häufigste Fall, beispielsweise

Typ	Was der Benutzer sieht und tun kann
	bei stapelbaren Modulen.
SelectAtLeastOne	Kontrollkästchen, aber der Nutzer muss mindestens eines aktivieren, um fortzufahren. Verhindert, dass der Nutzer mit leeren Händen davongeht.
SelectAll	Alle Optionen werden automatisch markiert und gesperrt: Der Benutzer kann sie nicht abwählen. Wird verwendet, um obligatorische Dateien bei deren Beschreibung darzustellen.

In einer Gruppe enthält `<plugins order="Explicit">` die Optionen, wobei jede eine `<plugin name="...">` ist (das Wort „Plugin“ bezeichnet hier eine Option des Assistenten, nicht eine .esp-Datei). Ihr Name ist die Bezeichnung, die der Benutzer ankreuzt. Eine Option besteht aus:

- `<description>`: Der Text, der rechts angezeigt wird, wenn die Option ausgewählt ist – beschreibe dort ihre Wirkung klar und deutlich;
- `<image path="..." />`: eine Abbildung, die zusammen mit der Beschreibung angezeigt wird (optional);
- `<conditionFlags>`: die Flags, die gesetzt werden, wenn die Option ausgewählt wird (§ 6.6);
- `<files>`: die Dateien, die installiert werden, wenn die Option ausgewählt wird;
- `<typeDescriptor>`: der Standardzustand der Option (§ 6.5).

6.5 Der Typ einer Option und ihr Verhalten

Jede Option verfügt über einen „Typ“, der ihren Standardzustand im Assistenten bestimmt: aktiviert oder nicht, bearbeitbar oder nicht. Dieser wird durch `<typeDescriptor>` beschrieben. Im einfachen Fall enthält er ein einzelnes `<type name="..." />`. Die fünf Werte und ihre Auswirkungen:

Typ	Auswirkung im Assistenten
Optional	Standardmäßig nicht markiert, kann beliebig markiert und entmarkiert werden. Der häufigste Fall.
Empfohlen	Standardmäßig aktiviert, kann vom Benutzer jedoch deaktiviert werden. Für eine empfohlene Auswahl.
Erforderlich	Aktiviert und gesperrt: Kann nicht deaktiviert werden. Für eine unverzichtbare Komponente.
Nicht verwendbar	Nicht markiert und gesperrt: Kann nicht markiert werden. Weist auf eine Option hin, die im aktuellen Kontext nicht verfügbar ist.
Könnte verwendbar	Kann aktiviert werden, ist jedoch als potenziell problematisch gekennzeichnet (oft mit einer Warnung).

Dynamische Typen: `<dependencyType>`. Der Typ einer Option kann sich je nach Bedingungen auch ändern. Anstelle eines festen `<type>` geben Sie einen `<dependencyType>` an, der einen `<defaultType name="..." />` (den Standardtyp, wenn keine Bedingung erfüllt ist) und eine Liste von `<pattern>` enthält: Jedes Muster ordnet Bedingungen einem `<type>` zu, der angewendet wird, wenn diese Bedingungen wahr sind.

Beispiel: Eine Option „ENB Ultra Preset“ kann standardmäßig „Optional“ sein, zu „Empfohlen“ werden, wenn der Benutzer dynamische Kerzen aktiviert hat, und zu „Nicht verwendbar“ werden, wenn eine

erforderliche Datei auf der Festplatte fehlt. Dieser Mechanismus macht einen Installer „intelligent“. Ein vollständiges Beispiel finden Sie in Anhang D.

6.6 Flags, Abhängigkeiten und bedingte Installationen

Diese drei Konzepte bilden das FOMOD-Bedingungssystem – das es einem Installationsprogramm ermöglicht, auf die Entscheidungen des Benutzers zu reagieren.

Flags – `<conditionFlags>` und `<flag>`. Ein Flag ist eine kleine Variable, die du setzt, wenn eine Option ausgewählt wird. Das `<conditionFlags>`-Tag einer Option enthält ein oder mehrere `<flag name="...">value</flag>`: „name“ ist der Name des Flags, und der Text zwischen den Tags ist sein Wert. So setzt `<flag name="res">4K</flag>` das „res“-Flag auf „4K“, sobald die Option aktiviert wird. Ein Flag hat für sich genommen keine sichtbare Wirkung: Es dient als Speicher, der später an anderer Stelle als Bedingung wiederverwendet wird.

Abhängigkeiten — `<dependencies operator="...">`. Eine Abhängigkeit ist eine zu prüfende Bedingung. Sie werden in `<dependencies>` gruppiert, deren „operator“-Attribut angibt, wie sie kombiniert werden sollen: „And“ (alle Bedingungen müssen wahr sein) oder „Or“ (eine einzige reicht aus). Es gibt zwei Arten von Bedingungen:

Bedingung	Was sie prüft
<code><flagDependency flag="..." value="..." /></code>	Wahr, wenn das genannte Flag genau den angegebenen Wert hat. Auf diese Weise reagieren Sie auf zuvor getroffene Entscheidungen (z. B.: Das „res“-Flag ist gleich „4K“).
<code><fileDependency file="..." state="..." /></code>	Wahr, abhängig vom Status einer Datei, die im Spiel des Benutzers bereits vorhanden ist oder nicht.

Das Attribut „state“ einer Dateibedingung kann drei Werte annehmen:

state	Bedeutung
Aktiv	Die Datei ist im Spiel vorhanden und aktiviert.
Inaktiv	Die Datei ist vorhanden, aber deaktiviert.
Fehlt	Die Datei fehlt.

Diese Abhängigkeiten dienen drei Zwecken: der Sichtbarkeit eines Schritts (in `<visible>`), dem dynamischen Typ einer Option (in einem `<pattern>` von `<dependencyType>`) und den nachfolgenden bedingten Installationen.

Bedingte Installationen – `<conditionalFileInstalls>`. Dieser Abschnitt, der nach den Schritten platziert wird, installiert Dateien nur dann, wenn bestimmte Bedingungen erfüllt sind, ohne dass der Benutzer etwas direkt auswählen muss. Er enthält `<pattern>`-Elemente, von denen jedes `<dependencies>` mit einem `<files>` verknüpft. Sind die Bedingungen des Musters am Ende des Assistenten erfüllt, werden die zugehörigen Dateien installiert.

Unterschied zur Sichtbarkeit von Schritten: `<visible>` entscheidet darüber, ob eine Seite angezeigt wird, während `<conditionalFileInstalls>` im Hintergrund darüber entscheidet, ob Dateien installiert werden. Typischer Fall: Ein Kompatibilitätspatch wird nur dann automatisch installiert, wenn der Benutzer zwei bestimmte Optionen aktiviert hat (deren Flags dann beide gesetzt sind).

Hinweis – Ein Flag muss zunächst durch eine Option (über `<conditionFlags>`) gesetzt werden, bevor es von einer Abhängigkeit geprüft werden kann. Definieren Sie daher Ihre Flags in den Schritten und prüfen Sie sie anschließend in den bedingten Mustern.

7. Beispiele für ModuleConfig.xml

Die folgenden Beispiele veranschaulichen die häufigsten Fälle. Sie entsprechen dem tatsächlich von XIMOD Architect erzeugten XML (die Einrückungen wurden zur besseren Lesbarkeit vereinfacht).

7.1 Minimalbeispiel – erforderliche Dateien

Ein Mod ohne Auswahlmöglichkeiten: Alle Dateien werden direkt installiert.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>My Simple Mod</moduleName>
  <requiredInstallFiles>
    <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
    <folder source="textures" destination="textures" priority="0"/>
  </requiredInstallFiles>
</config>
```

7.2 Exklusive Auswahl – SelectExactlyOne

Ein Schritt, der die Auswahl einer einzigen Texturauflösung ermöglicht; standardmäßig wird die 2K-Option empfohlen.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>HD Textures</moduleName>
  <moduleImage path="fomod\header.png"/>
  <installSteps order="Explicit">
    <installStep name="Texture resolution">
      <optionalFileGroups>
        <group name="Resolution" type="SelectExactlyOne">
          <plugins order="Explicit">
            <plugin name="2K">
              <description>2K textures (balanced).</description>
              <files>
                <folder source="2k" destination="textures" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Recommended"/>
              </typeDescriptor>
            </plugin>
            <plugin name="4K">
              <description>4K textures (high quality).</description>
              <files>
                <folder source="4k" destination="textures" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Optional"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
  </installSteps>
</config>
```

```

        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
</installSteps>
</config>

```

7.3 Bedingte Installation – Flags

Eine Option setzt ein Flag, wenn sie aktiviert ist; eine Datei wird nur installiert, wenn dieses Flag aktiv ist.

```

<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Conditional patch</moduleName>
  <installSteps order="Explicit">
    <installStep name="Options">
      <optionalFileGroups>
        <group name="Modules" type="SelectAny">
          <plugins order="Explicit">
            <plugin name="Module A">
              <description>Enables module A.</description>
              <conditionFlags>
                <flag name="ModuleA">0n</flag>
              </conditionFlags>
              <files>
                <folder source="A" destination="meshes" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Optional"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
  </installSteps>
  <conditionalFileInstalls>
    <patterns>
      <pattern>
        <dependencies operator="And">
          <flagDependency flag="ModuleA" value="0n"/>
        </dependencies>
        <files>
          <file source="patchA.esp" destination="patchA.esp" priority="0"/>
        </files>
      </pattern>
    </patterns>
  </conditionalFileInstalls>
</config>

```

Anhänge

Anhang A – Die Datei „Config.ini“

Die Konfiguration wird in der Datei „Config.ini“ gespeichert, die sich neben der ausführbaren Datei befindet. Hier ist ein vollständiges Beispiel:

```
; XIMOD Architect-Konfiguration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

Zu den bisherigen Schlüsseln kommen zwei weitere hinzu: „Country“ (ISO 3166-1 Alpha-3-Code des ausgewählten Landes) und „FirstStart“ (siehe § 2.5).

Die gespeicherte Geometrie der freien Fenster wird in einem zusätzlichen Abschnitt abgelegt:

```
[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650
```

Der Abschnitt [WindowPositions] merkt sich die Geometrie der freien Fenster (siehe § 2.6). Für jedes Fenster speichert WinPos_<id> seine Position (x,y) und WinSize_<id> seine Größe (Breite,Höhe). Diese Einträge werden erst erstellt, wenn das Fenster tatsächlich verschoben oder in der Größe geändert wurde; ohne sie öffnet sich das Fenster zentriert auf dem Hauptfenster. Die Größe des Hauptfensters wird in [Window] (WindowWidth / WindowHeight) gespeichert, seine Position jedoch nicht.

Anhang B – Tastaturkürzel

Unter macOS entspricht die Strg-Taste der Cmd-Taste.

Globale Tastenkürzel

Tastenkombination	Aktion
Strg+N	Neues Projekt.
Strg+O	Ordner öffnen.
Strg+Umschalt+O	Datei öffnen.
Strg+S	Speichern (FOMOD erstellen).
Strg+,	Einstellungen öffnen.
Strg+Q	Anwendung beenden.
F1	Das Fenster „Über“ anzeigen.

Fenster und Dialoge

Tastenkürzel	Aktion
Esc	Schließt das aktive freie Fenster (Länderauswahl, Eigenschaften, XML-Editor) oder den modalen Dialog (Einstellungen, Über, Bestätigung, Prüfbericht, Skripte). Bei einer Bestätigung wirkt sie wie „Abbrechen“. Keine Wirkung auf den Übersetzungseeditor und das Hauptfenster.

Listennavigation — Eigenschaften und Übersetzungseeditor

Taste	Aktion
↑ / ↓	Bewegt die Auswahl eine Zeile nach oben / unten.
Page Up / Page Down	Blättert um einen Bildschirm; die Schrittweite passt sich der Fensterhöhe an.
Home / End	Springt zur ersten / letzten Zeile (die letzte Zeile wird vollständig sichtbar gemacht).
Enter	Eigenschaften: wählt den markierten Eintrag aus. Übersetzungseeditor: setzt den Cursor in das Eingabefeld der Zeile.
Tab / Shift+Tab	Eigenschaften: nächste / vorherige Zeile (nie auf der Flagge). Übersetzungseeditor (während der Eingabe): nächstes / vorheriges Übersetzungsfeld.

Flaggenraster — Fenster „Länderauswahl“

Taste	Aktion
↑ ↓ ← → (8 2 4 6)	Bewegt den Auswahlrahmen im Raster.
Page Up / Page Down	Blättert um einen Bildschirm mit Flaggen.
Home / End	Springt zur ersten / letzten Flagge (die letzte Zeile wird vollständig sichtbar gemacht).
Tab / Shift+Tab	Nächste / vorherige Flagge.
Enter	Bestätigt das markierte Land.
Esc	Schließt das Fenster.

Anhang C – Glossar

Begriff	Definition
FOMOD	Standardformat für Mod-Installer bei Bethesda-Spielen, basierend auf den Dateien „info.xml“ und „ModuleConfig.xml“.
Schritt	Seite des Installationsassistenten, die eine oder mehrere Gruppen enthält.
Gruppe	Eine Reihe von Optionen (Plugins), die denselben Auswahltyp haben.
Plugin	Dem Benutzer angebotene Option, die mit den zu installierenden Dateien verknüpft ist.
Flag	Bedingungsflag (Paar aus Name und Wert), das zur Steuerung der Installation verwendet wird.
Endonym	Bezeichnung eines Landes oder einer Sprache, wie sie in dieser Sprache geschrieben wird.
ISO 639-3	Dreistelliger Sprachcode zur Benennung der Übersetzungsordner (z. B. fra, eng).
ISO 3166-1 Alpha-3	Dreistelliger Ländercode (z. B. FRA, USA).
Stammverzeichnis	Arbeitsordner, der die MOD-Dateien und den generierten Unterordner „fomod“ enthält.

Anhang D – Vollständiges FOMOD-Beispiel

Dieser Anhang stellt ein fiktives, aber vollständiges Projekt vor – „Dawnlight“, einen Beleuchtungs-Mod für Skyrim Special Edition –, das die gesamte Bandbreite der Möglichkeiten von XIMOD Architect in die Praxis umsetzt: erforderliche Dateien, mehrere Schritte, eine Bedingung für die Sichtbarkeit von Schritten, die fünf Gruppentypen, die fünf Optionstypen, Bedingungsflags, dynamische Optionstypen (Abhängigkeitsmuster) und bedingte Installationen. Die beiden folgenden Dateien bilden ein einziges Projekt und werden so wiedergegeben, wie XIMOD sie generiert (vereinfachte Einrückung).

D.1 info.xml

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
  <Name>Dawnlight</Name>
  <Author>Patrice</Author>
  <Version>1.2.0</Version>
  <Website>https://www.nexusmods.com/skyrimspedition/mods/98765</Website>
  <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
  <Groups>Visuals and Graphics</Groups>
  <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

In diesem Beispiel behandelte Funktionen:

- **Erforderliche Dateien:** requiredInstallFiles (eine Datei und ein Ordner).
- **Schritt 1:** Gruppe „SelectAll“ (Option „Required“) und „SelectExactlyOne“ (Optionen „Recommended“ / „Optional“, Flag „res“).
- **Schritt 2:** Gruppe „SelectAny“ (Flags „candles“ und „moon“) und „SelectAtMostOne“.
- **Schritt 3:** Sichtbarkeitsbedingung (<visible>), anschließend die Gruppe „SelectAtLeastOne“ mit einer Option vom Typ „dynamic“ (dependencyType: „Recommended“, wenn „candles=On“, „NotUsable“, wenn eine Datei fehlt) und einer Option „CouldBeUsable“.
- **Bedingte Installationen:** zwei Muster (Und- und Oder-Operatoren, Flag- und Dateibedingungen, Zustände „Active“ / „Inactive“ / „Missing“).

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
```

```

        <description>Scripts and meshes essential to the mod.</description>
        <files>
            <folder source="core\scripts" destination="scripts" priority="0"/>
        </files>
        <typeDescriptor>
            <type name="Required"/>
        </typeDescriptor>
    </plugin>
</plugins>
</group>
<group name="Texture resolution" type="SelectExactlyOne">
    <plugins order="Explicit">
        <plugin name="2K">
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
                <flag name="res">2K</flag>
            </conditionFlags>
            <files>
                <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
                <type name="Recommended"/>
            </typeDescriptor>
        </plugin>
        <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
                <flag name="res">4K</flag>
            </conditionFlags>
            <files>
                <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
                <type name="Optional"/>
            </typeDescriptor>
        </plugin>
    </plugins>
</group>
</optionalFileGroups>
</installStep>
<installStep name="Ambience options">
    <optionalFileGroups>
        <group name="Lighting effects" type="SelectAny">
            <plugins order="Explicit">
                <plugin name="Dynamic candles">
                    <description>Adds candles with flickering light.</description>
                    <conditionFlags>
                        <flag name="candles">0n</flag>
                    </conditionFlags>
                    <files>
                        <folder source="opt\candles" destination="meshes" priority="0"/>
                    </files>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>

```

```

        <typeDescriptor>
            <type name="Optional"/>
        </typeDescriptor>
    </plugin>
    <plugin name="Enhanced moon">
        <description>Increases the intensity of the moonlight.</description>
        <conditionFlags>
            <flag name="moon">0n</flag>
        </conditionFlags>
        <files>
            <folder source="opt\moon" destination="textures" priority="0"/>
        </files>
        <typeDescriptor>
            <type name="Optional"/>
        </typeDescriptor>
    </plugin>
</plugins>
</group>
<group name="Sound ambience" type="SelectAtMostOne">
    <plugins order="Explicit">
        <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
                <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
                <type name="Optional"/>
            </typeDescriptor>
        </plugin>
    </plugins>
</group>
</optionalFileGroups>
</installStep>
<installStep name="ENB compatibility">
    <visible>
        <dependencies operator="And">
            <flagDependency flag="res" value="4K"/>
            <fileDependency file="KnownConflict.esp" state="Inactive"/>
        </dependencies>
    </visible>
    <optionalFileGroups>
        <group name="ENB preset" type="SelectAtLeastOne">
            <plugins order="Explicit">
                <plugin name="Light ENB">
                    <description>Low-cost ENB preset.</description>
                    <files>
                        <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
                    </files>
                    <typeDescriptor>
                        <type name="Optional"/>
                    </typeDescriptor>
                </plugin>
                <plugin name="Ultra ENB">
                    <description>High-end ENB preset (dynamic type).</description>

```

```

    <files>
      <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
    </files>
    <typeDescriptor>
      <dependencyType>
        <defaultType name="Optional"/>
        <patterns>
          <pattern>
            <dependencies operator="And">
              <flagDependency flag="candles" value="0n"/>
            </dependencies>
            <type name="Recommended"/>
          </pattern>
          <pattern>
            <dependencies operator="Or">
              <fileDependency file="EnbHelper.dll" state="Missing"/>
            </dependencies>
            <type name="NotUsable"/>
          </pattern>
        </patterns>
      </dependencyType>
    </typeDescriptor>
  </plugin>
  <plugin name="Experimental extension">
    <description>Additional effects, not guaranteed.</description>
    <files>
      <folder source="enb\extra" destination="textures" priority="2"/>
    </files>
    <typeDescriptor>
      <type name="CouldBeUsable"/>
    </typeDescriptor>
  </plugin>
</plugins>
</group>
</optionalFileGroups>
</installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="0n"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="0n"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>

```

```
    <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
  </files>
</pattern>
</patterns>
</conditionalFileInstalls>
</config>
```

Anhang E – Lizenz und Danksagungen

Der Quellcode von XIMOD Architect wird unter der MIT-Lizenz (© 2025–2026) vertrieben. Die unten aufgeführten Komponenten von Drittanbietern unterliegen ihren eigenen Lizenzen; die vollständigen Details und Quellenangaben finden Sie in der Datei „THIRD-PARTY-NOTICES“, die im Lieferumfang der Anwendung enthalten ist.

Komponente	Quelle	Lizenz
XIMOD Architect-Code	—	MIT
Noto-Schriftarten	Google Fonts / Noto-Projekt	SIL Open Font License 1.1
Flaggen (SVG)	drapeauxdespays.fr	Lizenzgebührenfrei (einschließlich kommerzieller Nutzung)
ISO 3166-1 Alpha-3-Ländercodes	ISO — iso.org/obp	Freier Zugang
ISO 639-3-Sprachcodes	SIL — iso639-3.sil.org	Quellenangabe erforderlich (siehe unten)
Rust-Bibliotheken	crates.io -Ökosystem	MIT / Apache-2.0 (größtenteils)

ISO 639-3-Angabe: Die Sprachcodes stammen von iso639-3.sil.org (Quelle anzugeben). Die von XIMOD hinzugefügten Daten – Endonyme, zugehörige Schriftarten, Zuordnungen von Ländern zu Sprachen – sind nicht Bestandteil des ISO 639-3-Standards, und die Kennungen des Standards werden nicht verändert. Die Anwendung darf darüber hinaus keine Möglichkeit zur Weitergabe des vollständigen Codesatzes bieten.